

Install DCGM

Confirm that the target platform is supported, satisfy the applicable requirements, configure the NVIDIA package repository, and then use the package procedure for the installed Linux distribution.

Supported Platforms

Hardware and Environments

DCGM supports the following products and environments:

- All NVIDIA Kepler™ (K80) and newer NVIDIA datacenter (previously, NVIDIA Tesla®) GPUs
- NVIDIA® NVSwitch™ on NVIDIA DGX™ A100, NVIDIA HGX™ A100, and newer systems
- All NVIDIA Maxwell™ and newer non-datacenter GPUs, including NVIDIA® GeForce® and NVIDIA® Quadro® GPUs
- CUDA® 7.5 and later and NVIDIA Driver R450 and later
- Bare-metal systems and virtual machines with full GPU passthrough

On non-datacenter GPUs, some DCGM functionality is unavailable. The following table summarizes category-level availability. For the module organization and feature-specific documentation, see [Functionality by module](#).

Feature Availability by GPU Class

Feature availability by GPU class			
Feature area	Datacenter GPUs	Non-datacenter GPUs	Qualification
Field watches and GPU telemetry	Available	Available	Individual fields remain hardware- and driver-dependent.
Configuration management	Available	Available	Individual settings require corresponding GPU and driver support.
Passive health monitoring	Available	Available	Monitored subsystems depend on available fields and hardware.

Feature area	Datacenter GPUs	Non-datacenter GPUs	Qualification
Process and job statistics	Available	Available	Particular statistics can be unavailable when their source fields are unsupported.
Topology and link state	Available	Available	Reported PCIe, NVLink, and fabric details depend on system topology.
Host-engine introspection	Available	Available	Describes host-engine resource use rather than a GPU capability.
Policy notifications	Available	Not supported	Policy registration and notification are restricted to datacenter GPUs.
Single-node diagnostics	All levels supported for the applicable product	Level 1; higher levels only where explicitly supported	Plugin, SKU, driver, privilege, and installation requirements still apply.

Available means that the feature area is exposed for the GPU class. It does not guarantee that every field, setting, or diagnostic is supported on every GPU.

Linux Distributions

Linux Distributions and Architectures

Linux Distribution	x86_64	aarch64
Amazon Linux 2023	✓	✓
Azure Linux 3.0	✓	✓
Debian 12	✓	✓
Debian 13	✓	✓
Fedora 43	✓	✓
RHEL 8.y/Rocky Linux 8.y	✓	✓

Linux Distribution	x86_64	aarch64
RHEL 9.y/Rocky Linux 9.y	✓	✓
RHEL 10.y/Rocky Linux 10.y	✓	✓
SLES/OpenSUSE 15.y	✓	✓
SLES/OpenSUSE 16.y	✓	✓
Ubuntu 22.04 LTS	✓	✓
Ubuntu 24.04 LTS	✓	✓
Ubuntu 26.04 LTS	✓	✓

Requirements

The system-resource and NVIDIA software requirements apply when DCGM is run. They are not prerequisites for installing the development package.

System Resources

These requirements apply to bare-metal hosts, virtual machines, and containers.

Resource	Requirement
Minimum System Memory (Host RAM)	>= 16GB
Minimum CPU Cores	>= Number of GPUs

NVIDIA Software

Install a supported [NVIDIA Datacenter Driver](#). DCGM is tested with NVIDIA Datacenter Drivers; other driver types can omit functionality. The [NVIDIA driver documentation](#) describes the available branches and support timelines.

On systems with NVSwitch™ hardware, also install:

- Fabric Manager
- NVSwitch™ Configuration & Query (NSCQ) for Hopper and earlier GPUs
- NVSwitch™ Device Monitoring (NVSDM) for Blackwell and later GPUs

Fabric Manager, NSCQ, and NVSDM package major versions must match the NVIDIA driver major version. For example, driver version 565 uses `libnvidia-nscq-565` and `libnvdsdm-565`. Use `nvidia-smi` to identify the installed driver version.

See the [Fabric Manager installation instructions](#), the [NSCQ instructions in the HGX Software Guide](#), and the [NVSDM installation instructions](#).

Package Repository and Privileges

Configure the system package manager to use the NVIDIA repository for the installed Linux distribution. If the system uses a local CUDA repository package, update that package before installing DCGM. See the [CUDA Installation Guide for Linux](#) for repository setup instructions.

Package installation and system-service changes require root privileges. The examples use `sudo` to elevate only the command that requires them.

Commands are written for a Bash-compatible shell. A leading `$` is the prompt, not part of the command; backslashes continue a command on the next line.

Package Installation

Package Relationships

DCGM packages	
Package	Contents and relationships
<code>datacenter-gpu-manager-4-core</code>	CUDA-independent runtime components. Both the CUDA runtime package and the development package require the same version of this package.
<code>datacenter-gpu-manager-4-cuda<major></code>	CUDA-major-specific runtime components. This package requires <code>datacenter-gpu-manager-4-core</code> .
<code>datacenter-gpu-manager-4-proprietary*</code>	The subset of DCGM functionality that is not included in the DCGM open-source project . The relevant packages are installed as recommendations.
<code>datacenter-gpu-manager-4-multinode-cuda<major></code>	Optional multi-node diagnostic components, available for CUDA 12 and

Package	Contents and relationships
	later.
datacenter-gpu-manager-4-dev or datacenter-gpu-manager-4-devel	Headers, linker artifacts, CMake files, and samples. This package requires datacenter-gpu-manager-4-core ; it does not require a CUDA-major-specific runtime package.

The runtime-install commands below enable package recommendations. In addition to the

proprietary

 package family, the recommendation set contains non-DCGM packages such as the system NUMA library. The alternatives shown below omit all recommendations.

Select a CUDA Runtime Package

DCGM's CUDA-dependent binaries are divided into packages by CUDA user-mode driver major version. The package suffix identifies the target version; for example,

datacenter-gpu-manager-4-cuda12

 targets CUDA 12.

Use

nvidia-smi

 to identify the CUDA version supported by the installed driver:

```
$ nvidia-smi -q | grep -E 'Driver Version|CUDA Version'
Driver Version           : 575.57.08
CUDA Version             : 12.9
```

Set

CUDA_VERSION

 to the major version shown in the output. For the example above:

```
$ CUDA_VERSION=12
```

Warning

On Maxwell, Pascal, and Volta systems using driver version 580, set

CUDA_VERSION=12

. Although that driver supports the CUDA 13.0 user-mode API, CUDA Toolkit 13.0 does not support those GPU generations.

Stop an Existing DCGM Service

Stop DCGM before replacing an existing installation:

```
$ sudo systemctl list-unit-files nvidia-dcgm.service > /dev/null && \
sudo systemctl stop nvidia-dcgm
```

Install Packages

Use the section for the installed Linux distribution. Each procedure shows the CUDA-major runtime package, the development package, and the optional multi-node package. Install the packages needed on the system. After installing the runtime package, continue with [Start and Verify DCGM](#).

Amazon Linux 2023

1. Install the runtime package for the selected CUDA major version and its recommended packages.

```
$ CUDA_VERSION=<major version of CUDA user-mode driver>
$ sudo dnf install --assumeyes \
    --setopt=install_weak_deps=True \
    datacenter-gpu-manager-4-cuda${CUDA_VERSION}
```

To omit all weak dependencies, including recommended packages, replace

`--setopt=install_weak_deps=True` with `--setopt=install_weak_deps=False`.

2. If needed, install the optional multi-node diagnostic plugin.

```
$ sudo dnf install --assumeyes datacenter-gpu-manager-4-multinode-cuda${CUDA_VERSION}
```

3. If needed, install the development files.

```
$ sudo dnf install --assumeyes datacenter-gpu-manager-4-dev
```

Azure Linux 3.0

1. Install the runtime package for the selected CUDA major version and its recommended packages.

```
$ CUDA_VERSION=<major version of CUDA user-mode driver>
$ sudo tdnf install --assumeyes \
    --setopt=install_weak_deps=True \
    datacenter-gpu-manager-4-cuda${CUDA_VERSION}
```

To omit all weak dependencies, including recommended packages, replace

`--setopt=install_weak_deps=True` with `--setopt=install_weak_deps=False`.

2. If needed, install the optional multi-node diagnostic plugin.

```
$ sudo tdnf install --assumeyes datacenter-gpu-manager-4-multinode-cuda${CUDA_VERSION}
```

3. If needed, install the development files.

```
$ sudo dnf install --assumeyes datacenter-gpu-manager-4-dev
```

Ubuntu LTS and Debian

1. Remove any installations of the `datacenter-gpu-manager` and `datacenter-gpu-manager-config` packages

```
$ sudo dpkg --list datacenter-gpu-manager &> /dev/null && \  
sudo apt purge --yes datacenter-gpu-manager  
  
$ sudo dpkg --list datacenter-gpu-manager-config &> /dev/null && \  
sudo apt purge --yes datacenter-gpu-manager-config
```

2. Update the package registry cache

```
$ sudo apt-get update
```

3. Install the runtime package for the selected CUDA major version and its recommended packages.

```
$ CUDA_VERSION=<major version of CUDA user-mode driver>  
$ sudo apt-get install --yes \  
    --install-recommends \  
    datacenter-gpu-manager-4-cuda${CUDA_VERSION}
```

To omit all recommended packages, replace `--install-recommends` with `--no-install-recommends`.

4. If needed, install the optional multi-node diagnostic plugin.

```
$ sudo apt install --yes datacenter-gpu-manager-4-multinode-cuda${CUDA_VERSION}
```

5. If needed, install the development files.

```
$ sudo apt install --yes datacenter-gpu-manager-4-dev
```

RHEL, Rocky Linux, and Fedora

1. Remove any installations of the `datacenter-gpu-manager` and `datacenter-gpu-manager-config` packages.

```
$ sudo dnf list --installed datacenter-gpu-manager &> /dev/null && \  
sudo dnf remove --assumeyes datacenter-gpu-manager  
  
$ sudo dnf list --installed datacenter-gpu-manager-config &> /dev/null && \  
sudo dnf remove --assumeyes datacenter-gpu-manager-config
```

2. Update the package registry cache.

```
$ sudo dnf clean expire-cache
```

3. Install the runtime package for the selected CUDA major version and its recommended packages.

```
$ CUDA_VERSION=<major version of CUDA user-mode driver>  
$ sudo dnf install --assumeyes \  
    --setopt=install_weak_deps=True \  
    datacenter-gpu-manager-4-cuda${CUDA_VERSION}
```

To omit all weak dependencies, including recommended packages, replace

`--setopt=install_weak_deps=True` with `--setopt=install_weak_deps=False`.

4. If needed, install the optional multi-node diagnostic plugin.

```
$ sudo dnf install --assumeyes datacenter-gpu-manager-4-multinode-cuda${CUDA_VERSION}
```

5. If needed, install the development files.

```
$ sudo dnf install --assumeyes datacenter-gpu-manager-4-devel
```

SUSE SLES / OpenSUSE

1. Remove any installations of the `datacenter-gpu-manager` and `datacenter-gpu-manager-config` packages

```
$ sudo zypper search --installed-only --match-exact datacenter-gpu-manager &> /dev/null  
sudo zypper --non-interactive remove datacenter-gpu-manager  
  
$ sudo zypper search --installed-only --match-exact datacenter-gpu-manager-config &&  
sudo zypper --non-interactive remove datacenter-gpu-manager-config
```

2. Update the package registry cache

```
$ sudo zypper refresh
```

3. Install the runtime package for the selected CUDA major version and its recommended packages.

```
$ CUDA_VERSION=<major version of CUDA user-mode driver>
$ sudo zypper install --no-confirm \
    --recommends \
    datacenter-gpu-manager-4-cuda${CUDA_VERSION}
```

To omit all recommended packages, replace `--recommends` with `--no-recommends`.

4. If needed, install the optional multi-node diagnostic plugin.

```
$ sudo zypper install --no-confirm datacenter-gpu-manager-4-multinode-cuda${CUDA_VERSION}
```

5. If needed, install the development files.

```
$ sudo zypper install --no-confirm datacenter-gpu-manager-4-devel
```

Start and Verify DCGM

Enable the packaged systemd service at boot and start it now:

```
$ sudo systemctl enable --now nvidia-dcgm
$ systemctl is-active nvidia-dcgm
active
```

Use `dcgmi` to confirm that the host engine discovered the supported GPUs and NVSwitches in the system:

```
$ dcgmi discovery -l
8 GPUs found (Active).
...
6 NVSwitches found.
```

For local service options, environment changes, full unit replacement, and the multi-node diagnostics systemd generator, see [Customizing the systemd Service](#).

Continue with [the learning overview](#) to learn how the host engine represents and monitors the system.

Installed Components

Runtime Components

The runtime packages provide these principal components:

Runtime components		
Component	Installed item	Further information
DCGM shared library	<code>libdcgm.so.4</code>	Learn and the C/C++ API
DCGM modules	Loadable feature-module libraries	Modules and dcgm modules
Diagnostic plugins	Single-node diagnostic plugin libraries and supporting executables	Diagnostics and Diagnostic Plugins
NVIDIA Host Engine	<code>nv-hostengine</code>	nv-hostengine
systemd service	<code>nvidia-dcgm.service</code>	Customizing the systemd Service
Command-line interface	<code>dcgmi</code>	dcgmi
Profiling test utility	<code>dcgmproftester11</code> , <code>dcgmproftester12</code> , or <code>dcgmproftester13</code> , according to the installed CUDA-major package	dcgmproftester
Python bindings	<code>/usr/share/datacenter-gpu-manager-4/bindings/python3/</code>	Installed Python modules and examples
Multi-node diagnostics	Components supplied by the optional <code>datacenter-gpu-manager-4-multinode-cuda<major></code> package	Multi-Node Diagnostics

Development Components

The development package adds the following files to the runtime components installed by its `datacenter-gpu-manager-4-core` dependency:



[Privacy Policy](#) | [Your Privacy Choices](#) | [Terms of Service](#) | [Accessibility](#) | [Corporate Policies](#) | [Product Security](#) | [Contact](#)

Copyright © 2018-2026, NVIDIA Corporation.

Last updated on 2026-09-10.